

RELIEFLINK – LIVE DISASTER COORDINATION PLATFORM

**A project report submitted in fulfilment of
the requirements for the award of the degree of
MASTER OF COMPUTER APPLICATIONS**

From

APJ Abdul Kalam Kerala Technological University



Submitted By

VIVEK (MEA25MCA-2027)



MEA Engineering College

Department of Computer Applications

Vengoor P.O, Perinthalmanna, Malappuram, Kerala-679325

OCTOBER 2026

DEPARTMENT OF COMPUTER APPLICATIONS

MEA ENGINEERING COLLEGE

PERINTHALMANNA-679325



CERTIFICATE

This is to certify that the Project Report entitled "RELIEFLINK – LIVE DISASTER COORDINATION PLATFORM" is a bonafide record of the work done by Vivek (MEA25MCA-2027) under our supervision and guidance. The report has been submitted in fulfillment of the requirement for the award of the Degree of Master of Computer Applications from the APJ Abdul Kalam Kerala Technological University for the year 2026.

Mr. Sajeesh M Mr. Sajeesh M

Assistant Professor Assistant Professor

Project Guide Head of the Department

Dept. of Computer Applications Dept. of Computer Applications

Acknowledgements

First and foremost, I would like to thank the Almighty God for giving me the knowledge, strength, and perseverance to successfully complete this project.

An endeavor undertaken over a long period can be accomplished only with the guidance and encouragement of many well-wishers. I take this opportunity to express my sincere gratitude to all those who supported and encouraged me throughout the completion of this project.

I express my deep sense of gratitude to our respected Principal, Dr. J. Hussain, for his inspiration and for creating an excellent academic environment in the college that enabled me to undertake this project successfully.

I would like to extend my heartfelt thanks to Mr. Sajeesh M., Assistant Professor and Head of the Department of Computer Applications, for providing the necessary permission and facilities to carry out this project in a systematic manner. I am highly indebted to Mr. Sajeesh M., Assistant Professor, Department of Computer Applications, for his valuable guidance, timely advice, constant encouragement, and wholehearted support throughout the successful completion of this project.

I also express my sincere gratitude to Mr. Adil Narakkoden, Assistant Professor, Department of Computer Applications, Project Coordinator, for his continuous encouragement and moral support during the completion of this project.

Vivek (MEA25MCA-2027)

DATE:

RELIEFLINK – LIVE DISASTER COORDINATION PLATFORM

ReliefLink is a full-stack disaster management web application developed to connect affected people, volunteers, and NGOs/Relief Centers in real time. The platform coordinates SOS alerts, rescue and relief tasks, resource allocation, and tracks the complete disaster response lifecycle from the moment an emergency is reported to its final resolution.

Affected persons can log in using phone OTP verification and send emergency SOS alerts with their exact GPS location, mark themselves as "I'm Safe", request essential relief items such as food, water, and medicine, and use an interactive map-based Camp Finder to locate the nearest active relief shelter. Volunteers register their specific skills, accept nearby rescue or delivery tasks, and update task status in real time while navigating to incident locations.

NGOs and Relief Centers act as the command layer of the system – monitoring all SOS alerts on a live map, assigning them to the nearest available and most suitable volunteer, managing camp capacity and inventory, approving relief requests, and broadcasting disaster alerts to affected regions.

ReliefLink goes beyond a basic CRUD application by incorporating a Live Incident Timeline, automatic Nearest-Available-Responder matching using MongoDB's geospatial queries, Volunteer Skill Matching, Live Shelter Capacity tracking, Partial Fulfillment and Cross-NGO Resource Sharing, and a centralized Resource Allocation Board that highlights unmet needs across the network.

Overall, ReliefLink integrates real-time communication, geolocation intelligence, secure authentication, and coordinated resource management into a single platform to make disaster response faster, more transparent, and more effective.

TABLE OF CONTENTS

INTRODUCTION

Disaster response is one of the most time-critical and coordination-heavy activities faced by any society. During floods, fires, earthquakes, and other emergencies, the speed and accuracy with which affected people are located, resources are allocated, and volunteers are dispatched can directly determine the outcome of the response. Traditional disaster management relies heavily on manual coordination through phone calls, radio communication, and physical control rooms, which often leads to delays, miscommunication, and inefficient use of available resources.

ReliefLink is a smart, real-time disaster coordination platform developed to digitalize and streamline this process. It provides a common platform connecting Affected People, Volunteers, and NGOs/Relief Centers. Affected persons can send SOS alerts with their exact location, request relief items, and check on nearby shelters, while volunteers and relief organizations coordinate the response from a live, map-based command interface.

The system uses MongoDB's geospatial querying capability to automatically identify the nearest available volunteer for an incident and matches volunteers to tasks based on their registered skills. Relief camps display live occupancy and capacity, allowing both affected persons and coordinators to make informed decisions about where help is available.

NGOs and Relief Centers can monitor all SOS alerts on a live map, assign tasks to suitable volunteers, manage inventory and donations, track partial fulfillment of relief requests, transfer surplus resources to other NGOs facing shortages, and broadcast emergency alerts to specific regions.

The application is developed using the MERN-style stack, consisting of React (Vite) for the frontend, Node.js and Express.js for the backend, and MongoDB with Mongoose for data storage. Zustand is used for frontend state management, Leaflet.js provides interactive mapping, Socket.io enables real-time updates, and JWT together with Phone OTP verification secures authentication.

The main aim of ReliefLink is to make disaster response more efficient, transparent, and coordinated by integrating real-time communication, geolocation intelligence, skill-based task matching, and centralized resource management into a single platform.

SYSTEM ANALYSIS

System analysis is the process of studying the existing disaster response process, identifying its limitations, and defining the requirements for developing an improved system.

In ReliefLink, the analysis focuses on the interaction between affected people, volunteers, and NGOs/Relief Centers during an emergency.

The existing disaster response process has several limitations, including unverified or delayed SOS reporting, manual volunteer dispatch, limited visibility of shelter capacity, poor tracking of relief inventory, and difficulty in coordinating between multiple relief organizations.

ReliefLink addresses these limitations by providing a centralized real-time platform where SOS alerts can be reported, automatically matched to the nearest suitable volunteer, tracked through a live incident timeline, and resolved with full visibility into resource availability.

Existing System

In the traditional disaster response system, affected people generally depend on phone calls, word-of-mouth, or physical visits to report emergencies. Coordination between volunteers and relief organizations is largely manual, and matching the right responder to the right incident can be slow.

The major limitations of the existing system are:

- Lack of real-time tracking of SOS alerts and rescue operations.
- Delayed or inefficient volunteer dispatch.
- No automated way of finding the nearest available responder.
- Limited visibility of shelter occupancy and capacity for affected people.
- Poor tracking of relief inventory and donations across multiple NGOs.
- Difficulty matching volunteers to tasks that require specific skills.
- Lack of a centralized platform connecting affected people, volunteers, and relief organizations.

Proposed System

ReliefLink is proposed as a real-time digital solution to overcome the limitations of the existing disaster response process. Affected people can submit SOS alerts with their exact geographical location and request relief items directly from the platform.

The system uses MongoDB's geospatial (\$geoNear) capability to automatically identify the nearest available volunteer, and matches volunteers to tasks based on their registered skills. NGOs and Relief Centers can monitor incidents on a live map, assign tasks, manage inventory and camps, and share surplus resources with other NGOs facing shortages.

The proposed system also provides a Live Incident Timeline, Live Shelter Capacity

indicators, and a Resource Allocation Board to give coordinators complete visibility into ongoing operations.

Module Description

ReliefLink consists of three primary user modules along with supporting real-time coordination modules.

Affected Person Module

The Affected Person Module allows users to:

- Register and log in securely using Phone OTP verification.
- Send emergency SOS alerts with exact GPS coordinates.
- Mark themselves as "I'm Safe" to notify responders and family.
- Request essential relief items such as food, water, and medicine.
- Use an interactive map-based Camp Finder to locate the nearest active shelter.
- View live weather updates and disaster alerts.

Volunteer Module

The Volunteer Module allows volunteers to:

- Log in using email and password.
- Register specific skills such as First Aid, Swimming, or Medical support.
- View nearby SOS requests and accept rescue or delivery tasks.
- Update task status in real time (Accepted, En Route, Completed).
- Navigate to incident locations or relief camps using route directions.
- View individual performance metrics.

NGO / Relief Center Module

The NGO/Relief Center Module provides:

- A command center to monitor all SOS alerts on a live map and assign volunteers.
- Inventory management for tracking food, water, and medicine stock.
- Camp management for updating capacity, occupancy, and facilities.
- Approval and fulfillment of relief item requests, including partial fulfillment.
- Cross-NGO resource transfer for surplus inventory.
- Broadcast of disaster alerts and push notifications to specific regions.

Real-Time Coordination Module

This module powers the platform's live coordination features, including the Live Incident Timeline, Nearest Available Responder matching, Volunteer Skill Matching, Live Shelter Capacity tracking, and the Resource Allocation Board that highlights unfulfilled requests across the network.

Notification Module

ReliefLink uses Socket.io for real-time updates related to SOS status, volunteer dispatch, and relief fulfillment, ensuring all stakeholders receive instant updates without manually refreshing the application.

Sprint Lifecycle

The development of ReliefLink follows an Agile-based development approach, where the project is divided into smaller development cycles called sprints. Each sprint focuses on implementing and testing a specific set of features.

A typical sprint lifecycle consists of:

1. Sprint Planning – Identify the features and tasks to be completed.
2. Requirement Analysis – Analyse the requirements of the selected features.
3. Development – Implement the planned functionality.
4. Testing – Test the implemented features and identify errors.
5. Review – Evaluate the completed work.
6. Feedback and Improvement – Make necessary improvements based on the review.
7. Next Sprint – Continue development with the next set of requirements.

This approach allows ReliefLink to be developed incrementally and makes it easier to test and improve individual modules throughout the development process.

The sprint-wise task breakdown followed during development is given below:

Sprint 1 – Affected Person Module

Task	Pending Task	Hour of Completion	Expected Date	Actual Date	Reason for Delay
Login/ Registration (Phone OTP)	-	3 hr.	05/07/2026	05/07/2026	-
SOS Alert Page	-	4 hr.	08/07/2026	08/07/2026	-
Camp Finder Map	-	3 hr.	10/07/2026	10/07/2026	-
Relief Request Page	-	3 hr.	13/07/2026	13/07/2026	-
Safety Status & Alerts	-	2 hr.	15/07/2026	15/07/2026	-

Sprint 2 – Volunteer Module

Task	Pending Task	Hour of Completion	Expected Date	Actual Date	Reason for Delay
Volunteer Login Page	-	1 hr.	18/07/2026	18/07/2026	-
Skill Profile Setup	-	2 hr.	20/07/2026	20/07/2026	-
Nearby SOS/Task List	-	3 hr.	22/07/2026	22/07/2026	-
Live Task Tracking & Navigation	-	3 hr.	25/07/2026	25/07/2026	-
Performance Summary Page	-	1 hr.	27/07/2026	27/07/2026	-

Sprint 3 – NGO / Relief Center Module

Task	Pending Task	Hour of Completion	Expected Date	Actual Date	Reason for Delay
NGO/Admin Login Page	-	1 hr.	30/07/2026	30/07/2026	-
Live SOS Command Map	-	4 hr.	02/08/2026	02/08/2026	-
Volunteer/Task Assignment	-	3 hr.	05/08/2026	05/08/2026	-
Inventory & Camp Management	-	3 hr.	08/08/2026	08/08/2026	-
Resource Allocation Board	-	3 hr.	10/08/2026	10/08/2026	-
Broadcast & Alerts Module	-	2 hr.	11/08/2026	11/08/2026	-

FEASIBILITY STUDY

Feasibility study is carried out to determine whether the proposed ReliefLink system is practical and suitable for development and implementation. It evaluates the project from economic, technical, operational, behavioural, software, and hardware perspectives.

Economical Feasibility

ReliefLink is economically feasible because it is developed using widely available open-source technologies and frameworks. The system uses React, Node.js, Express.js,

MongoDB, and Leaflet.js, which reduces development and licensing costs. Cloud-based services and external APIs are used only where required, keeping overall development cost relatively low.

Technical Feasibility

The project is technically feasible because all the required technologies are well-established and suitable for building a real-time coordination platform. The system uses:

- React (Vite) with Zustand for the frontend and state management.
- Node.js and Express.js for backend development.
- MongoDB and Mongoose for database management, including geospatial queries.
- Leaflet.js for interactive map-based location reporting and tracking.
- Socket.io for real-time communication.
- JWT and Phone OTP verification for authentication and security.

These technologies provide the necessary capabilities to implement the proposed system.

Operational Feasibility

ReliefLink is operationally feasible because the system provides separate interfaces for affected people, volunteers, and NGO/Relief Center coordinators. Affected persons can easily send SOS alerts and locate shelters, volunteers can view and accept tasks, and coordinators can manage the entire response from a centralized command dashboard. Therefore, the system can support the real-time operational demands of disaster response.

Behavioural Feasibility

The system is designed to encourage quick and confident participation from all stakeholders during a crisis. A simple SOS process, live incident timelines, and clear shelter/resource visibility make the system accessible and trustworthy for affected people, volunteers, and relief organizations alike.

Software Feasibility

The required software environment is readily available for development and deployment. ReliefLink is based on a modern JavaScript stack and uses supporting libraries and services for mapping, real-time communication, authentication, and geospatial analysis. The project follows a structured architecture with separate frontend and backend components, controllers, models, routes, middleware, and utility modules.

Hardware Feasibility

ReliefLink does not require specialized hardware for normal operation. The system can be accessed through computers or smartphones with an internet connection. A basic development or user device with a modern web browser is sufficient. Volunteers and affected persons can use mobile devices to send alerts, view maps, and navigate to locations even during field operations.

SOFTWARE ENGINEERING PARADIGM

Software engineering paradigm defines the approach and methodology followed for developing a software system. ReliefLink follows an Agile Software Development Life Cycle (SDLC) approach. Agile development focuses on incremental development, continuous testing, regular feedback, and improvement of the system.

The Agile approach is suitable for ReliefLink because the system consists of multiple modules such as SOS handling, volunteer coordination, NGO management, inventory tracking, and real-time notifications. These modules can be developed and tested incrementally.

Agile SDLC Model

The Agile SDLC model divides the software development process into small and manageable iterations. Instead of developing the complete system at once, individual features are implemented, tested, reviewed, and improved during successive development cycles.

The major stages followed are:

1. Requirement Gathering – Identify the requirements of affected people, volunteers, and NGOs.
2. Planning – Select the features to be implemented in each iteration.
3. Design – Design the database, interfaces, system architecture, and workflows.
4. Development – Implement the selected features.
5. Testing – Test each completed feature and fix identified errors.
6. Review – Evaluate the developed functionality.
7. Feedback and Improvement – Apply necessary changes and proceed to the next iteration.

This iterative process helps in identifying problems early and allows new requirements or improvements to be incorporated during development.

Scrum Framework

Scrum is an Agile framework used to organize and manage software development through short development cycles called Sprints. For ReliefLink, the development is organized into different sprints based on system functionality, with each sprint focusing on completing a specific group of features.

Scrum Activities

- Sprint Planning: Features and tasks for the upcoming sprint are selected.
- Sprint Development: Developers implement the selected functionality.
- Testing: Developed features are tested for correctness and reliability.
- Sprint Review: Completed work is evaluated.
- Sprint Retrospective: Problems encountered during the sprint are analysed and improvements are identified.
- Next Sprint: The next set of features is planned and developed.

Using Scrum helps organize the development of ReliefLink systematically and supports continuous improvement throughout the project.

SYSTEM REQUIREMENT SPECIFICATION

System Requirement Specification (SRS) describes the software and hardware resources required for the development and operation of the ReliefLink system. It defines the minimum environment necessary to run the application effectively.

Software Requirements

The following software and technologies are required for developing and running ReliefLink:

Category	Requirement
Frontend	React 18, Vite
State Management	Zustand
Styling	Vanilla CSS (custom utility system)
Backend	Node.js, Express.js
Database	MongoDB, Mongoose
Real-time Communication	Socket.io
Maps	Leaflet.js
Authentication	JWT, Phone OTP Verification
Development Environment	Visual Studio Code

Browser	Modern web browser such as Chrome, Edge or Firefox
---------	--

ReliefLink uses React with Zustand for the frontend, Node.js and Express.js for the RESTful backend, and MongoDB with Mongoose for storing application data, including geospatial indexes. Leaflet.js provides interactive mapping, while Socket.io powers real-time coordination.

Hardware Requirements

ReliefLink does not require specialized hardware. A standard computer or smartphone with internet connectivity is sufficient for accessing the application.

Minimum Hardware Requirements

Component	Requirement
Processor	Dual-Core processor or above
RAM	4 GB minimum
Storage	10 GB free space
Display	1280 x 720 resolution or above
Network	Stable Internet connection
Mobile Device	Android/iOS smartphone for field operations

Volunteers and affected persons can use smartphones to send SOS alerts, view live maps, and navigate to locations during field operations.

SYSTEM DESIGN

System design describes the overall structure, components, database, user interactions, and data flow of the ReliefLink system. It defines how the different modules interact with each other and how information is stored and processed.

ReliefLink follows a modular architecture consisting of the Affected Person Module, Volunteer Module, NGO/Relief Center Module, Real-Time Coordination Module, and Notification Module. The system uses MongoDB for data storage and a Node.js/Express.js backend to provide RESTful APIs to the React frontend.

Database Design & Normalization

ReliefLink uses MongoDB with Mongoose as its database system. MongoDB is a NoSQL database that stores data in document-based collections, and Mongoose is used to define schemas and interact with the database.

The major collections used in the system are:

- User – Stores affected person, volunteer, NGO, and admin details.
- SosRequest – Stores emergency incident details, location, priority, and status.
- Task – Stores volunteer assignments linked to SOS or relief requests.
- ReliefRequest – Stores requests for essential items and their fulfillment status.
- ReliefCamp – Stores shelter details, location, and capacity.
- Inventory – Stores stock details for each NGO/camp.
- DisasterAlert – Stores system-wide broadcast warnings.
- MissingPerson – Stores lost-and-found registry details.
- Donation – Stores monetary and goods donation records.
- SafetyStatus & Notification – Store safety updates and in-app notifications.

The database design separates different types of information into individual collections, reducing unnecessary duplication and making the system easier to maintain and query using geospatial indexes.

Database Schemas & Data Dictionary

The major data models of ReliefLink are as follows:

User

Field	Description
name, email, phone, password	Core authentication fields
role	affected, volunteer, ngo, or admin
otp, isVerified	Phone OTP verification details
profileImage, address, district, state	Profile fields
location	GeoJSON Point (longitude, latitude)
skills, vehicleType, isAvailable, tasksCompleted, rating	Volunteer-specific fields
organizationName, website	NGO-specific fields
isSafe, pushSubscription, isActive	Status fields

SosRequest

Field	Description
disasterType, description, numberOfPeople, medicalEmergency	Incident details
priority	low, medium, high, or critical
status	pending, assigned, in_progress, or resolved
location, address	GeoJSON Point of the incident
assignedVolunteer, assignedAt, resolvedAt, notes	Assignment and tracking details

Task

Field	Description
volunteerId	Reference to the assigned volunteer
type	rescue, delivery, medical, etc.
relatedSos / relatedRelief	Reference to the related SOS or relief request
status, priority	assigned, accepted, in_progress, completed
startLocation, destination	GeoJSON Points for routing
acceptedAt, startedAt, completedAt	Timestamps

ReliefRequest

Field	Description
userId, location	Requesting user and their location
items	Array of { category, quantity, fulfilledQuantity }
status, priority	pending, approved, in_delivery, delivered
approvedByNGO, assignedVolunteer	Reference to responsible NGO and volunteer

ReliefCamp

Field	Description
name, managedBy, description, facilities	Basic camp details
location, address, district	Camp location details
capacity, currentOccupancy, acceptingRefugees	Capacity metrics

status	active, full, or closed
--------	-------------------------

Inventory

Field	Description
campId, ngold	Ownership references
itemName, category, quantity, unit, minStockLevel	Item stock details
expiryDate, donor, lastUpdated	Metadata

DisasterAlert, MissingPerson, Donation, SafetyStatus & Notification

DisasterAlert stores title, message, type, and target regions for broadcast warnings. MissingPerson stores name, age, gender, photo, last seen location/date, and status (missing, found, reunited). Donation stores the donor NGO, camp, type (monetary or goods), amount/items, and status. SafetyStatus logs a user's safety updates over time, and Notification stores in-app messages with read status.

UML Design

UML (Unified Modeling Language) is used to represent the structure and behaviour of the ReliefLink system. The UML design helps describe the interaction between different users and system components.

The major entities involved in the system include:

- Affected Person
- Volunteer
- NGO / Relief Center
- SOS Request
- Task
- Relief Request
- Relief Camp
- Inventory
- Disaster Alert
- Missing Person

The UML diagrams provide a visual representation of how these entities interact with the

system.

Use Case Diagram

The Use Case Diagram represents the major activities performed by the three primary users.

Affected Person

- Register/Login via Phone OTP
- Send SOS Alert
- Mark Self as Safe
- Request Relief Items
- Find Nearest Camp
- View Weather & Disaster Alerts
- Report Missing Person

Volunteer

- Login
- Register Skills
- View Nearby SOS/Tasks
- Accept Task
- Update Task Status
- Navigate to Location
- View Performance Metrics

NGO / Relief Center

- Login
- Monitor Live SOS Map
- Assign Volunteers/Tasks
- Manage Inventory
- Manage Camps
- Approve/Fulfill Relief Requests
- Transfer Resources to Other NGOs
- Broadcast Alerts
- View Resource Allocation Board

System Scenarios

Scenario 1 – Affected Person Sends SOS

1. Affected person logs in using Phone OTP.
2. Affected person sends an SOS alert with current location.
3. System determines priority based on disaster type and details.
4. Nearest available volunteer is identified using \$geoNear.
5. Volunteer is notified and assigned to the SOS.
6. Live incident timeline begins tracking the response.

Scenario 2 – Volunteer Completes a Task

1. Volunteer views nearby SOS/relief tasks matching their skills.
2. Volunteer accepts the task and navigates to the location.
3. Volunteer updates status to En Route, then Completed.
4. NGO verifies the completed task.
5. Incident timeline is updated and the case is marked resolved.

Scenario 3 – NGO Allocates Resources

1. NGO reviews pending relief requests on the Resource Allocation Board.
2. NGO checks available inventory against requested items.
3. If stock is insufficient, partial fulfillment is recorded.
4. NGO requests surplus resources from another NGO if needed.
5. Relief items are dispatched and the request status is updated.

Data Flow Diagrams (DFD Level 0, 1, 2)

DFD Level 0

Figure 6.1: Data Flow Diagram - Level 0

DFD Level 1

Figure 6.2: Data Flow Diagram - Level 1

DFD Level 2

Figure 6.3: Data Flow Diagram - Level 2

SYSTEM DEVELOPMENT

System development is the process of converting the system design and requirements into

a working software application. ReliefLink is developed as a full-stack real-time web application. The frontend, backend, database, mapping services, and real-time communication components are integrated to provide the required functionality.

The project follows a modular structure, with separate components for authentication, SOS handling, volunteer coordination, NGO/inventory management, and real-time notifications.

Coding & Subsystem Architecture

ReliefLink follows a client-server architecture. The React frontend provides the user interface, while the Node.js and Express.js backend handles business logic and REST API requests. MongoDB is used for storing application data, including geospatial indexes.

Figure 7.1: Subsystem Architecture of ReliefLink

Frontend Subsystem

The frontend is developed using React 18 with Vite, using Zustand for state management. It contains reusable components and separate pages for different user roles.

Important frontend components include:

- `SosAlertWidget.jsx` – Emergency SOS trigger and status widget.
- `LiveMap.jsx` – Leaflet-based live incident and camp map.
- `IncidentTimeline.jsx` – Displays the live timeline of an SOS.
- `ResourceAllocationBoard.jsx` – Displays shortages across the network.

The major pages include:

- Home
- Login / OTP Verification
- Register
- Affected Person Dashboard
- Volunteer Dashboard
- NGO/Admin Command Center
- Camp Finder
- SOS Detail
- Inventory & Donations

Backend Subsystem

The backend is developed using Node.js and Express.js. It provides RESTful APIs for authentication, SOS handling, task management, relief requests, and NGO operations.

The backend consists of:

- Controllers – Handle application business logic.
- Models – Define MongoDB data structures with geospatial fields.
- Routes – Define API endpoints.
- Middleware – Handle authentication, authorization, and file uploads.
- Utils – Provide geospatial matching and skill-matching services.

Authentication and Authorization

ReliefLink uses JWT-based authentication for volunteers and NGOs, and Phone OTP verification for affected persons. Role-based authorization ensures that affected people, volunteers, and NGOs can access only the features appropriate to their roles.

Geospatial Matching Subsystem

This subsystem uses MongoDB's \$geoNear operator to automatically identify the nearest available volunteer for an SOS request, and filters candidates further using registered skills to ensure the right responder is matched to the right incident.

Database Subsystem

MongoDB stores user information, SOS requests, tasks, relief requests, camps, inventory, alerts, missing persons, and donations. Mongoose is used to define schemas, including 2dsphere geospatial indexes, and communicate with the database.

Map and Location Subsystem

The system uses Leaflet.js to allow affected persons to send their exact location during an SOS, display live shelter capacity on the map, and let volunteers navigate to incident locations or relief camps.

Real-Time Communication Subsystem

Socket.io is used to provide real-time updates for SOS status changes, volunteer dispatch, and incident timeline events, allowing users to receive updates instantly without manually refreshing the application.

Resource Coordination Subsystem

This subsystem manages inventory tracking, partial fulfillment of relief requests, and cross-NGO resource transfers, feeding data into the centralized Resource Allocation Board.

Overall Development Flow

The major development flow of ReliefLink is:

User → React Frontend (Zustand) → Express REST API / Socket.io → Controller → MongoDB (Geospatial Queries) → Response → React Frontend

Figure 7.2: Overall System Development Flow

SYSTEM TESTING AND IMPLEMENTATION

System testing is an important phase of software development used to verify whether the ReliefLink system works according to the specified requirements. Testing helps identify errors, verify individual functionalities, and ensure that the complete system operates correctly under real-time conditions.

ReliefLink is tested across its major modules, including authentication, SOS handling, geospatial matching, task management, inventory and camp management, and notifications.

Types of Testing & Test Case Suite

Unit Testing

Unit testing verifies individual components or functions of the application separately.

Examples include:

- Testing OTP generation and verification.
- Testing SOS priority calculation.
- Testing \$geoNear volunteer matching logic.
- Testing inventory quantity updates.

Integration Testing

Integration testing verifies whether different modules communicate correctly with each other. Examples include:

- Frontend and backend API communication.
- Backend and MongoDB geospatial query communication.
- SOS module and task assignment integration.
- Relief request and inventory/fulfillment integration.

System Testing

System testing evaluates the complete ReliefLink application as a single integrated system.

The complete workflow from SOS submission to incident resolution is tested to ensure all modules work together correctly.

Functional Testing

Functional testing verifies whether each system function produces the expected result.

Test Case Suite

Test Case ID	Test Case	Expected Result	Status
TC01	Affected Person Registration (OTP)	User account is created and verified successfully	Pass
TC02	User Login	User is authenticated successfully	Pass
TC03	Invalid OTP/Login	Error message is displayed	Pass
TC04	SOS Alert Submission	SOS is created with correct location and priority	Pass
TC05	Nearest Volunteer Matching	Closest available volunteer is identified correctly	Pass
TC06	Skill-Based Task Matching	Volunteer with required skill is matched to task	Pass
TC07	Task Acceptance	Task status updates to Accepted for the volunteer	Pass
TC08	Live Task Status Update	Status change reflects in real time via Socket.io	Pass
TC09	Camp Finder	Nearest active relief camp is displayed correctly	Pass
TC10	Relief Request Submission	Relief request is created with requested items	Pass

Test Case ID	Test Case	Expected Result	Status
TC11	Partial Fulfillment	Remaining quantity is tracked correctly after partial delivery	Pass
TC12	Cross-NGO Resource Transfer	Inventory is updated correctly for both NGOs	Pass
TC13	Inventory Update	Stock levels update after donation/usage	Pass
TC14	Camp Capacity Update	Occupancy percentage updates on the live map	Pass
TC15	Missing Person Report	Report is registered and searchable	Pass
TC16	Broadcast Alert	Alert is delivered to users in the targeted region	Pass

System Implementation

System implementation is the process of deploying the developed ReliefLink modules into a working application environment. The implementation consists of the following major

components:

Frontend Implementation

The frontend is implemented using React 18, Vite, and Zustand. It provides separate interfaces for affected people, volunteers, and NGO/Relief Center coordinators.

Figure 8.1: ReliefLink Home Page

Affected Person Dashboard

The Affected Person Dashboard allows users to send SOS alerts, view their safety status, and access relief requests and nearby camps.

Figure 8.2: Affected Person Dashboard

Live SOS Map

The live map displays active SOS alerts, relief camps, and their capacity, updated in real time using Socket.io.

Figure 8.3: Live SOS Command Map

Volunteer Dashboard

The volunteer dashboard displays nearby tasks matched to the volunteer's skills, along with navigation and status update options.

Figure 8.4: Volunteer Dashboard

NGO / Admin Command Center

The NGO dashboard provides an overview of SOS alerts, volunteer assignments, inventory, and camp status.

Figure 8.5: NGO Command Center

Inventory & Resource Allocation Board

Coordinators can track stock levels, donations, and unfulfilled requests across the network from a centralized board.

Figure 8.6: Resource Allocation Board

Incident Timeline

Every SOS generates an automated, chronological timeline showing key events from submission to resolution.

Figure 8.7: Live Incident Timeline

The implemented system integrates the frontend, backend, database, geospatial matching, mapping, and notification mechanisms into a unified disaster coordination platform.

SYSTEM MAINTENANCE

System maintenance is the process of modifying, updating, and improving a software system after its implementation. Maintenance is necessary to ensure that ReliefLink continues to operate reliably during real disaster scenarios.

The maintenance of ReliefLink involves monitoring the application, correcting errors, updating software components, maintaining the database, and improving system performance under high-load, emergency conditions.

Corrective Maintenance

Corrective maintenance is performed to identify and fix errors or bugs that may occur during system operation. Examples include:

- Fixing OTP and authentication errors.
- Correcting SOS submission or geolocation issues.
- Resolving volunteer-matching inaccuracies.
- Fixing notification delivery errors.
- Correcting database-related errors.

Adaptive Maintenance

Adaptive maintenance involves modifying the system to work with changes in the operating environment. Examples include:

- Updating dependencies and libraries.
- Supporting newer browser and mobile OS versions.
- Updating map and geolocation service integrations.
- Adapting to changes in cloud or database services.

Perfective Maintenance

Perfective maintenance improves the existing features and user experience of the system. Possible improvements include:

- Improving dashboard and live-map performance.
- Enhancing the volunteer-matching algorithm.

- Improving analytics for coordinators.
- Adding new relief-tracking options.

Preventive Maintenance

Preventive maintenance is carried out to reduce the possibility of future system failures. It includes:

- Regular database backup.
- Monitoring system performance during high-load emergency events.
- Updating security dependencies.
- Reviewing authentication and authorization mechanisms.
- Maintaining proper database and geospatial index structure.

Figure 9.1: System Maintenance Process

Regular maintenance helps ensure that ReliefLink remains secure, reliable, and capable of supporting the changing requirements of affected people, volunteers, and relief organizations.

FUTURE ENHANCEMENT

ReliefLink provides a complete platform for coordinating disaster response. However, the system can be further enhanced with additional features and technologies to improve its efficiency, scalability, and reach.

The possible future enhancements of the system are:

Mobile Application

A dedicated Android and iOS mobile application can be developed for affected people and volunteers, providing easier access to SOS reporting, task management, and offline-capable navigation.

AI-Based Damage Assessment

Machine learning models can be used to analyse satellite or drone imagery to automatically assess disaster damage and prioritize response in the worst-affected areas.

Drone-Based Reconnaissance

Integration with drone feeds can help volunteers and NGOs assess hard-to-reach incident locations before dispatching ground teams.

Predictive Disaster Analytics

Historical incident and weather data can be analysed to predict high-risk zones and pre-position resources before a disaster occurs.

Offline-First Mode

Since disasters often disrupt network connectivity, an offline-first mode with local data caching and automatic sync can be introduced for critical SOS and task features.

Blockchain-Based Donation Tracking

A blockchain ledger can be introduced to provide fully transparent and verifiable tracking of monetary and goods donations.

Government & Emergency Services Integration

ReliefLink can be integrated with government disaster management agencies and emergency services for wider reach and official coordination.

Multilingual Support

Additional regional languages can be introduced to make the system accessible to a wider range of affected communities.

Advanced Notifications

Future versions can provide SMS and push notifications for users without constant internet access, ensuring critical alerts are never missed.

Cloud Scalability

The system can be further optimized for large-scale, multi-region deployment using scalable cloud infrastructure, load balancing, and distributed services.

These enhancements can make ReliefLink more intelligent, resilient, and suitable for large-scale, real-world disaster management operations.

CONCLUSION

ReliefLink is a real-time disaster coordination platform developed to improve the speed, efficiency, and transparency of emergency response. The system provides a common platform for affected people, volunteers, and NGOs/Relief Centers, allowing the complete disaster response process to be handled digitally and in real time.

Affected persons can send SOS alerts with their exact location and request relief items,

while the system automatically identifies the nearest available and most suitable volunteer using geospatial and skill-based matching. This reduces the dependency on manual coordination and significantly speeds up response times.

NGOs and Relief Centers can monitor incidents on a live map, assign volunteers, manage inventory and camps, and coordinate resource sharing across organizations. Volunteers can access their assigned tasks, navigate to incident locations, and update progress in real time, providing a clear and trackable workflow from SOS submission to final resolution.

The system also introduces advanced coordination features such as a Live Incident Timeline, Live Shelter Capacity tracking, and a Resource Allocation Board that gives coordinators complete visibility into ongoing operations and unmet needs.

ReliefLink combines modern technologies such as React, Node.js, Express.js, MongoDB's geospatial capabilities, and Socket.io real-time communication to create an integrated disaster coordination platform.

Overall, ReliefLink demonstrates how technology and real-time coordination can be combined to create a more efficient, transparent, and life-saving approach to disaster management. The system provides a strong foundation that can be further expanded with AI-based damage assessment, drone integration, predictive analytics, and large-scale cloud deployment.

APPENDIX

The Appendix contains supporting materials related to the development and implementation of the ReliefLink system. It provides additional technical information that supports the main content of the project documentation.

1. Project Directory Structure

The ReliefLink project is organized into separate frontend and backend directories. The backend contains configuration files, controllers, middleware, models, routes, and utility functions, while the frontend contains components, pages, state stores, and styling files.

Figure 12.1: ReliefLink Project Directory Structure

2. Important Backend Components

The backend contains the following major components:

- Controllers – Handle authentication, SOS, tasks, relief requests, and NGO operations.
- Models – Define database schemas such as User, SosRequest, Task, ReliefRequest, ReliefCamp, Inventory, DisasterAlert, MissingPerson, Donation, SafetyStatus, and

Notification.

- Routes – Define API routes for different system operations.
- Middleware – Handles JWT authentication, role authorization, and file uploads.
- Utils – Contains the geospatial matching and skill-matching functionality.

Figure 12.2: Backend Folder Structure

3. Important Frontend Components

The frontend contains reusable components and separate pages for the different user roles. Important components include the SOS alert widget, live map, incident timeline, and resource allocation board.

Figure 12.3: Frontend Folder Structure

4. Sample Application Screens

The following screenshots can be included as supporting evidence of the implemented system:

Figure 12.4: Affected Person Dashboard

Figure 12.5: SOS Alert Page

Figure 12.6: Live SOS Command Map

Figure 12.7: Volunteer Dashboard

Figure 12.8: Inventory & Resource Allocation Board

Figure 12.9: Camp Finder

Figure 12.10: Live Incident Timeline

5. API and Project Execution

The ReliefLink backend can be started from the backend directory using the development command, while the frontend can be started separately from the frontend directory. The project also provides an optional database seed command for creating initial demo data.

BIBLIOGRAPHY

The following technologies, frameworks, libraries, and services were used in the development of the ReliefLink project:

1. React 18 – Frontend development framework.

2. Vite – Frontend build and development tool.
3. Zustand – Frontend state management library.
4. Node.js – Backend runtime environment.
5. Express.js – Backend web application framework.
6. MongoDB – NoSQL database management system with geospatial support.
7. Mongoose – MongoDB object modelling library.
8. Leaflet.js – Interactive map library.
9. Socket.io – Real-time communication.
10. JWT – Authentication and authorization.
11. Phone OTP Services – Phone-based verification for affected persons.
12. Vanilla CSS (Custom Utility System) – User interface styling.